1. Java Fundamentals
1.1 Java Virtual Machine (JVM) – The JVM interprets compiled Java bytecode and enables Java applications to run on any device or OS.
1.2 Java Runtime Environment (JRE) – JRE provides libraries and the JVM to run Java applications but does not include tools for development.
1.3 Java Development Kit (JDK) – JDK includes JRE and development tools like compiler (javac) used for building Java applications.
1.4 Platform Independence in Java – Java's compiled bytecode runs on any system with a JVM, making it platform-independent.
1.5 Just-In-Time (JIT) Compiler – JIT compiles bytecode to native code at runtime, improving performance.

2. Java Program Structure
2.1 Main Method Syntax (public static void main(String[] args)) – This is the entry point for any Java program.
2.2 Role of Keywords: public, static, void, main, String[] args – public allows access, static allows execution without an object, void means no return, main is the method name, String[] args is an array for command-line arguments.
2.3 What if main is not static? – The JVM can't call it without creating an object, so it results in an error.

3. Java Memory and String Handling
3.1 Java String Pool – A special memory area to store common string literals for memory optimization.
3.2 System.out vs System.err – System.out is for standard output; System.err is for error messages.

4. Input in Java
4.1 Ways to Take Input
Command Line – Input passed while executing the program.
BufferedReader – Efficient reader for character input, usually from the console.
Console Class – Used for secure password and command-line input.
Scanner Class – A simple class to read input of various types (int, float, string).

5. Java Syntax and Operators
5.1 print, println, and printf – print prints inline, println adds a newline, printf formats output.
5.2 >> vs >>> – >> is arithmetic right shift, >>> is logical right shift.
5.3 Dot Operator . – Used to access members of a class or object.

6. Data Types in Java
6.1 Primitive Data Types – byte, short, int, long, float, double, char, boolean.
6.2 Non-Primitive Data Types – Strings, Arrays, Classes, Interfaces.
6.3 Wrapper Classes – Convert primitives to objects (e.g., Integer, Double).
Need and Features – Useful in collections, support nulls, utility methods.
Autoboxing and Unboxing – Automatic conversion between primitives and wrapper objects.

7. Variables in Java
7.1 Instance vs Local Variables – Instance variables belong to objects; local variables exist within methods.
7.2 Instance vs Class Variables – Class variables are static and shared among all objects.
7.3 static Keyword – Indicates a variable/method belongs to the class, not an instance.
7.4 this Keyword – Refers to the current object.

8. String Handling Classes
8.1 String vs StringBuffer – String is immutable; StringBuffer is mutable and thread-safe.
8.2 StringBuffer vs StringBuilder – Both are mutable; StringBuilder is faster but not thread-safe.

9. Object-Oriented Programming Concepts
9.1 Overview – Java is based on OOP: classes, objects, and principles.
9.2 Encapsulation – Binding data and methods together, restricting direct access.
9.3 Inheritance – Acquiring properties from parent class.
9.4 Polymorphism – One task can be performed in many ways.
9.5 Abstraction – Hiding internal details and showing essential features.

10. Classes and Objects
10.1 What is a Class? – A blueprint for creating objects.
10.2 What is an Object? – An instance of a class.

10.3 Real-Life Example – Class: Car, Object: Toyota Camry.

11. Constructors in Java
11.1 What is a Constructor? – A block called when an object is created.
11.2 Types of Constructors
Default – No-argument constructor provided by Java if none is defined.
Parameterized – Accepts parameters to initialize object values.
Constructor Overloading – Multiple constructors with different parameters.
11.3 Copy Constructor – Copies values from another object (user-defined).
11.4 Constructor vs Method – Constructor initializes an object; method performs actions.
11.5 What if No Constructor is Provided? – Java provides a default one.

12. Packages in Java
12.1 Definition and Purpose – Packages group related classes and avoid name conflicts.

13. Pointers in Java
13.1 Why Java Does Not Support Pointers – For safety, simplicity, and portability.

14. Interfaces in Java
14.1 What is an Interface? – A contract that defines abstract methods.
14.2 Features – All methods are abstract by default; supports multiple inheritance.
14.3 Example (Remote Control Analogy) – Interface: Remote; Implementation: TV, AC.
14.4 Default Methods – Interfaces can now have default methods with body.
14.5 Interface vs Abstract Class – Interfaces are pure abstraction; abstract classes may have implementation.

15. Abstraction
15.1 What is Abstraction? – Hiding internal logic and exposing functionality.
15.2 Abstract Class in Java – Can have both abstract and concrete methods.
15.3 Abstract vs Concrete Methods – Abstract: no body; Concrete: with implementation.
15.4 Real-Life Example (Vehicle) – Abstract vehicle class; concrete: Bike, Car.

16. Encapsulation
16.1 What is Encapsulation? – Data wrapping using private variables and public getters/setters.
16.2 Data Hiding and Controlled Access – Prevent unauthorized access to fields.
16.3 Real-Life Example (Capsule Analogy) – Medicine capsule: content hidden inside.

17. Aggregation
17.1 What is Aggregation? – A relationship where one class uses another (has-a).
17.2 Has-a Relationship – E.g., Department has Students.
17.3 Unidirectional Association – One-way relationship from one class to another.

18. Inheritance
18.1 What is Inheritance? – Allows one class to acquire fields and methods of another.
18.2 Types: Single, Multilevel, Hierarchical, Multiple (via Interfaces only) – Java supports multiple inheritance using interfaces.
18.3 Why Multiple Inheritance is Not Supported – To avoid ambiguity.
18.4 Diamond Problem – Occurs with multiple inheritance; solved using interfaces.

19. Polymorphism
19.1 What is Polymorphism? – Ability of object to take many forms.
19.2 Compile-Time (Method Overloading) – Same method name with different parameters.
19.3 Runtime (Method Overriding) – Subclass defines its version of a superclass method.
19.4 Real-Life Example (Printer) – Print: PDF, Doc, Image (different implementations).
19.5 Static Methods and Overriding – Static methods are not overridden, they are hidden.

20. Collection Framework
20.1 Overview – Provides classes and interfaces for storing data efficiently.
20.2 Interfaces: Set, List, Queue, Deque – Define data structures and behavior.
20.3 Classes: ArrayList, Vector, LinkedList, TreeSet, etc. – Implement data structures.

21. Vector in Java
21.1 Features – Dynamic array, thread-safe.
21.2 Vector vs ArrayList – Vector is synchronized; ArrayList is faster.
21.3 Synchronization – Vector ensures thread safety with synchronization.

22. Exception Handling
22.1 What is Exception Handling? – Process of handling runtime errors.
22.2 try, catch, finally, throw – try: monitor code, catch: handle exception, finally: cleanup, throw: raise exception.
22.3 Types: Built-in, User-defined – Java has many built-in; custom exceptions can be created.
22.4 Runtime Exceptions – Occur during execution (e.g., NullPointerException).
22.5 Checked vs Unchecked – Checked: compile-time, Unchecked: runtime.
22.6 Error vs Exception – Errors are serious problems, exceptions are recoverable.
22.7 NullPointerException – Occurs when accessing object members on a null reference.

23. final Keyword
23.1 Final Variable – Cannot be changed once assigned.
23.2 Final Method – Cannot be overridden by subclass.
23.3 Final Class – Cannot be inherited.
23.4 Purpose and Usage – For constant values and immutability.

24. finally Block
24.1 What is it? – A block that always executes after try-catch.
24.2 Usage in try-catch-finally – Used to close resources.
24.3 Resource Cleanup – Clean up files, DB connections, etc.
24.4 Example – Closing file streams.

25. finalize() Method
25.1 What is it? – Called by GC before object is removed.
25.2 Role in Garbage Collection – Used to release resources.
25.3 Usage and Limitations – Not reliable; Java 9 deprecated it.
25.4 Example – Cleanup code in finalize() method.

26. super Keyword
26.1 What is super? – Refers to parent class.
26.2 Access Parent Constructor – Use super() to call parent constructor.
26.3 Access Parent Methods – Invoke overridden methods.
26.4 Access Parent Variables – Reference parent class variables.
26.5 Real-life Use – To reuse parent behavior.

27. Multitasking
27.1 What is Multitasking? – Performing multiple tasks at once.
27.2 Threads – Smallest unit of a multitasking program.
27.3 Benefits – Efficient CPU usage, faster execution.

28. Multithreading
28.1 What is a Thread? – A lightweight process.
28.2 Multithreaded Program – Program using multiple threads.
28.3 Advantages – Better performance, responsiveness.
28.4 Threads vs Processes – Threads share memory; processes do not.

29. Thread Creation
29.1 Extending Thread Class – Override run() method.
29.2 Implementing Runnable – Implement Runnable interface.
29.3 Benefits of Runnable – Allows extending another class.
29.4 run() vs start() – start() starts new thread; run() is just a method call.

30. Thread Lifecycle
30.1 New – Thread created.
30.2 Runnable – Ready to run.
30.3 Blocked – Waiting for monitor lock.

30.4 Waiting – Waiting for signal.
30.5 Terminated – Thread has finished.

31. Thread Methods
31.1 suspend() – Deprecated; freezes thread.
31.2 resume(), stop() and Alternatives – Also deprecated; use wait() and notify().
31.3 Main Thread – Initial thread in Java application.

32. Garbage Collection
32.1 Why Needed – To free memory.
32.2 JVM Memory Management – JVM handles memory allocation/deallocation.
32.3 finalize() Role – Cleanup before deletion.
32.4 Avoiding Memory Leaks – Use proper cleanup and weak references.

33. JDBC
33.1 What is JDBC? – Java API for connecting to databases.
33.2 Components: API, DriverManager, Connection, Statement, ResultSet, RowSet – Provide ways to interact with DB.
33.3 JDBC Drivers and Types – Types: JDBC-ODBC, Native-API, Network, Thin driver.
33.4 Architecture – Client → JDBC Driver → DB.
33.5 Steps to Connect – Load driver → Get connection → Execute query → Process results → Close connection.
33.6 Code Example – Use Connection, Statement, ResultSet.

34. JDBC Interfaces and Classes
34.1 Connection – Connects to DB.
34.2 Statement – Executes SQL.
34.3 ResultSet – Holds returned data.
34.4 RowSet Types: JdbcRowSet, CachedRowSet, WebRowSet, FilteredRowSet, JoinRowSet – Provide flexible data handling.

35. == vs equals()
35.1 == Operator (Reference) – Compares object references.
35.2 equals() Method (Content) – Compares content of objects.
35.3 String Example – "abc" == new String("abc") is false; equals() is true.
35.4 Use Cases – Use == for primitives, equals() for objects.

36. Project: Bank Management System
36.1 Problem Solved – Manage banking operations like accounts, transactions.
36.2 Reason for Selection – Covers CRUD, OOP, JDBC, exception handling.
36.3 Project Structure – Layered: UI → Service → DAO → DB.
36.4 Tools and Libraries – Java, JDBC, MySQL, IDE.
36.5 System Workflow – Login → Account → Deposit/Withdraw/Transfer.
36.6 Security & DB Interaction – Use input validation, encryption, and SQL handling.